Taller Practico Integrador: QA en Produccion

Estrategia de Pruebas

Plataforma Web "CineFlex"

Integrantes:

Integrante 1 (Backend): _____

Integrante 2 (Frontend y UI): _____

Integrante 3 (Postman/Newman y JMeter): _____

Integrante 4 (Estrategia, CI y Documentacion): _____

Docente: _____

Institucion: _____

Fecha: ____ / ____ / _____

Introduccion

CineFlex es una plataforma web de venta de boletos de cine compuesta por tres capas: un Frontend (UI) donde el usuario inicia sesion, explora la cartelera, selecciona asientos y paga; un Backend (API REST en Node.js/Express) que gestiona autenticacion, inventario de salas y logica de reservas; y una capa de Base de Datos y Terceros que incluye una pasarela de pagos externa (simulada en el entorno de pruebas).

Este documento define la estrategia de aseguramiento de calidad aplicada al proyecto. Antes de escribir codigo de automatizacion, se documentan las decisiones arquitectonicas de pruebas: el enfoque de integracion, la estrategia de sistema y regresion, y la comparativa entre Selenium WebDriver y Cypress. Su objetivo es garantizar que CineFlex funcione correctamente y no colapse en la noche de su gran estreno.

Objetivos

Objetivo general

Definir e implementar una estrategia integral de pruebas (integracion, sistema, regresion y rendimiento) que valide el comportamiento funcional y no funcional de la plataforma CineFlex sin depender de los servicios bancarios reales.

Objetivos especificos

- Seleccionar y justificar el enfoque de integracion adecuado para un escenario con una pasarela de pagos externa no disponible.
- Disenar un Stub que simule la respuesta del banco para probar el flujo de ventas de extremo a extremo.
- Definir requisitos funcionales y no funcionales medibles, con su respectivo plan de prueba.
- Establecer un criterio de seleccion de pruebas de regresion que optimice recursos sin omitir riesgos criticos.
- Comparar tecnicamente Selenium WebDriver y Cypress y justificar su uso dentro del proyecto.

Actividad A. Pruebas de Integracion (Stubs y Drivers)

Enfoque seleccionado: Top-Down

Para el escenario de CineFlex se selecciona el enfoque de integracion Top-Down (de arriba hacia abajo). En este enfoque, los modulos de nivel superior se prueban primero y los modulos de nivel inferior aun no disponibles se sustituyen temporalmente por Stubs.

- El Frontend (interfaz de pago) y el Backend (endpoint de transaccion) ya estan terminados.
- El unico componente que falta es el mas bajo de la jerarquia: la pasarela de pagos del banco.
- Top-Down permite validar hoy el flujo completo (UI -> API -> pago) reemplazando solo el banco por un Stub.
- El enfoque Bottom-Up habria exigido probar primero la pasarela real (indisponible) y habria requerido Drivers para simular al Backend, lo cual es innecesario porque el Backend ya existe.

Diseno del Stub de la pasarela de pagos

El Stub se implementa dentro del backend como una funcion procesarPago() que simula la respuesta del banco de forma determinista segun los ultimos cuatro digitos del numero de tarjeta. No se conecta a ningun servicio externo y siempre devuelve una de las siguientes respuestas predefinidas:

- Pago exitoso (APROBADO): devuelve un identificador de transaccion y el codigo APROBADO.
- Tarjeta rechazada (RECHAZADO): devuelve el motivo de rechazo del banco emisor.
- Respuesta invalida: devuelve una respuesta malformada o inesperada de la pasarela.
- Error del servicio: simula que la pasarela esta caida o fuera de tiempo.

Integracion del Stub con el backend

El endpoint POST /api/reservas ejecuta la logica de negocio (validar pelicula, validar asientos y validar tarjeta) y luego invoca al Stub en lugar de llamar a la API real del banco. Cuando el banco otorgue acceso a su entorno de pruebas, solo sera necesario sustituir la llamada al Stub por la llamada real, sin modificar el resto del flujo. De esta forma, CineFlex procesa correctamente las ventas sin depender del banco real.

Escenarios de prueba del Stub

Escenario	Estimulo (tarjeta)	Respuesta del Stub	Resultado esperado en CineFlex
Pago exitoso	4242 4242 4242 4242	APROBADO + transaccionId	201: reserva confirmada y ticket emitido
Tarjeta rechazada	4000 0000 0000 0002	RECHAZADO	402: sin reserva y sin ticket
Respuesta invalida	4000 0000 0000 1234	Respuesta malformada	502: error controlado de pasarela
Error del servicio	4000 0000 0000 0000	Pasarela no disponible	503: error controlado de pasarela

Como se comprueba el procesamient

Se ejecuta la coleccion de Postman y la sui
verifica que unicamente en el escenario de p
un ticket valido (codigo con formato CF-XXX
Esto demuestra que CineFlex procesa corre
real.

Actividad B. Pruebas de Siste

Requisitos funcionales

- RF1 - Autenticacion: El usuario debe pod
token JWT; el sistema debe bloquear la cuent
- RF2 - Compra de boletos: El usuario debe
libres y completar la compra con pago aproba

Requisitos no funcionales (medibles

- RNF1 - Rendimiento (SLA): El endpoint d
de 300 ms en condiciones normales. Se verifi

coleccion Postman.

- RNF2 - Capacidad: El endpoint de consul
concurrentes con una tasa de error menor al
JMeter (500 hilos, ramp-up 15 s).

Como probar cada requisito

RF1 y RF2 se prueban con pruebas de s
exitoso, bloqueo, flujo de compra, ticket) y co
HTTP y aserciones). RNF1 se mide con la a
mide con el plan de carga de JMeter, analiza
Report.

Estrategia de regresion: cambio de co

Caso analizado: un desarrollador cambia uni
perfil de usuario.

Criterio de seleccion: se aplica regresion
cosmetico y aislado en la vista de perfil, por
pagos. Se ejecutan:

- Pruebas de humo (smoke) de la vista de
- Pruebas de regresion de la navegacion g
(para detectar regresiones de renderizado o e
- No serian prioritarias: las pruebas de la p
pruebas de carga, porque no comparten depe

Justificacion: el riesgo de que un cambio de
la suite seria un desperdicio de recursos. Si
de humo de autenticacion y navegacion po
error en CSS podria propagarse a otras vist
seria un riesgo, por lo que se mantienen las p

Actividad C. Comparativa: Se

Que es cada herramienta

Selenium WebDriver es un framework de au
real a traves del protocolo W3C WebDriver,
es una herramienta de testing de fronten
aplicaciones JavaScript modernas, con sopor

Tabla comparativa tecnica

Criterio	Selenium WebDriver	Cypress
Arquitectura	Cliente-servidor fuera del navegador (WebDriver)	Se ejecuta dentro del navegador

Navegadores

Chrome, Edge, Firefox, Safari, etc.

	Principalmente Chromium (Firefox experimental)

Lenguajes

Java, JS, Python, C#, Ruby, etc.

Solo JavaScript/TypeScript

Multiples pestanas/
origenes

Soporte nativo

Limitado

Control de red (mocks)

Requiere herramientas externas

Integrado (interceptores)

Velocidad de ejecucion

Menor (arranca navegador real)

Mayor (integrado al navegador)

Complejidad de configuracion

Requiere driver por navegador

Configuracion minima

Escenarios de CineFlex para cada herram

- Selenium WebDriver: se utiliza para validar el f
cartelera -> asientos -> pago -> ticket) contra el na
la navegacion protegida (cierre de sesion y boton A
- Cypress: seria conveniente para pruebas de co
ejemplo validar la logica del formulario de pago o c
simuladas, aprovechando sus mocks de red y su v

Por que Selenium es obligatorio en este taller: por
multiplataforma sobre un navegador real, aplicand
comportamiento real del navegador (incluida la na
limita a un entorno integrado. Esto permite al doc
POM, que es el criterio explicito de la rubrica.

Conclusiones

- El enfoque Top-Down con un Stub de la pasare
CineFlex hoy, sin depender del banco real.
- La definicion de requisitos funcionales y no fun
objetiva la calidad del sistema.
- La regresion selectiva basada en riesgo optimi
- Selenium WebDriver es la herramienta adecua
multiplataforma; Cypress es una alternativa comple
componentes.

Referencias

- Selenium WebDriver - documentacion oficial: h
- Cypress - documentacion oficial: https://www.c
- Apache JMeter - documentacion oficial: https:/
- Postman / Newman - documentacion oficial: ht
- Page Object Model - Selenium docs: https://ww